

S I G N A L

xNIDS Console — a working demo platform

Explaining Deep Learning-based Network Intrusion Detection Systems for Active Intrusion Response

Paper analysis · Gap identification · **LSTM-Autoencoder** · Full-stack implementation

Reference paper: **xNIDS** — Wei, Li, Zhao, Hu · USENIX Security 2023

github.com/CactiLab/code-xNIDS

DL-NIDS detect well. They explain almost nothing.

- **Semantic gap**

Detection results are scores or labels. Operators need to know which host, flow, or feature to act on — and the model doesn't say.

- **High error cost**

A single missed attack lets an intruder in; a flood of false positives buries operators in manual triage.

- **Opaque architecture**

DL-NIDS use history-dependent, structurally complex inputs (e.g. IP headers) that existing explainers were never built for.

WHY ACTIVE RESPONSE STALLS

Network traffic



Deep Learning model



Detection score: 0.93



??? — no actionable reason

xNIDS: explain the black box, then act on it

① History approximation

Finds a small set of relevant history inputs (not just x_t , not all of $X_{[t-\delta, t]}$) that reproduce the original detection score within a tolerance δ .

② Decay-weighted sampling

Synthesizes samples around history inputs via Weighted Random Sampling, biased toward the most recent windows — matching how LSTM forget-gates discount old inputs.

③ Sparse group lasso

Groups structurally related features (e.g. TCP sub-fields) so the surrogate model includes/excludes whole groups — capturing dependencies LIME/SHAP/IG ignore.

④ Defense rule generation

Explanation results → rule scope (per-flow / per-host / multi-hosts) → block strategy (passive / assertive / aggressive) → unified rule → OpenFlow, iptables, Pfsense, Squid.

xNIDS outperformed five baselines on every axis

4

target DL-NIDS evaluated

Kitsune · ODDS · RNN-IDS · AE-IDS

0.32

best fidelity AUC (lower=better)

vs 0.41–0.86 for LIME/SHAP/LEMNA/IG/LRP

>0.8

stability score across systems

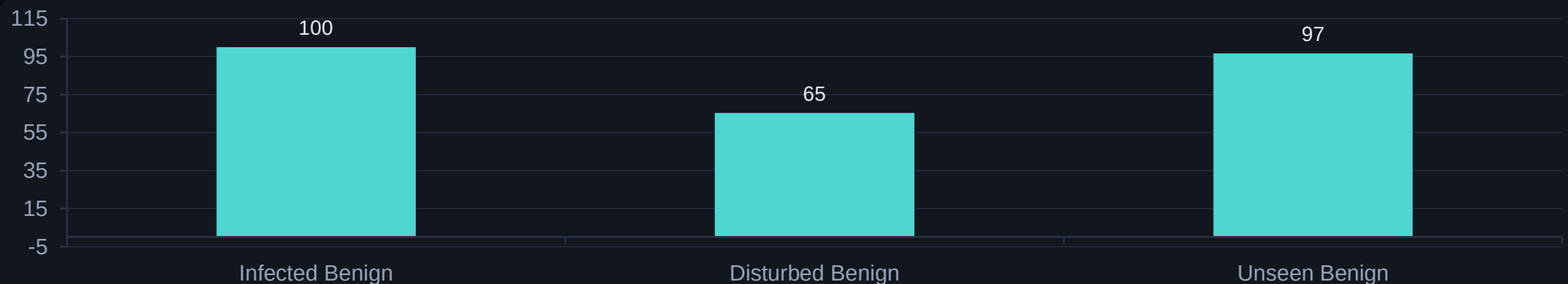
vs 0.36–0.64 for best baselines

~25ms

avg. defense rule generation

across OpenFlow, iptables, Pfsense

Troubleshooting impact (paper Table 11) — false positives eliminated after patching:



The explainer carries a burden the detector should share

WHAT THE PAPER ACKNOWLEDGES

- **All 4 target systems treat history shallowly**

Kitsune: fixed sliding window. RNN-IDS/AE-IDS: minimal sequence modeling. ODDS: LSTM-based but still opaque to the explainer.

- **History-awareness is bolted on, not built in**

xNIDS must repeatedly sample around approximated history inputs to recover what the model already implicitly used.

- **Cost: latency and bounded fidelity**

Up to ~900ms per explanation (paper Fig. 5); fidelity is capped by how well sampled windows stand in for true temporal reasoning.

OUR RESPONSE

Give the detector native history-awareness instead of asking the explainer to reconstruct it from outside.

An LSTM-Autoencoder reconstructs the **entire history sequence** end-to-end, exposing per-timestep, per-feature error as a free, gradient-free importance signal — complementing (not replacing) the xNIDS-style explainer.

LSTM-Autoencoder: built from scratch, history-native

ARCHITECTURE

History sequence (5×23 features)



Encoder LSTM ($23 \rightarrow 12$ hidden)



Latent hidden-state sequence



Decoder LSTM ($12 \rightarrow 23$)



Reconstructed sequence + per-step error

WHY THIS ARCHITECTURE

Native history modeling —

reconstructs all 5 history steps, not just x_t , directly addressing Ch1.

Unsupervised, benign-only —

consistent with the paper's own target systems; no labeled attack data required to train.

Free importance signal —

decay-weighted per-feature error needs zero sampling, cutting explanation latency.

Zero heavy DL dependency —

full gate math + BPTT implemented in NumPy; trained with real gradient descent.

From signal to response: the full pipeline



FastAPI · SQLAlchemy (SQLite / MySQL) · NumPy + scikit-learn · React 19 + Vite + recharts

One request, full pipeline: POST /api/attacks/simulate → detect → explain → generate rule → persist → render — under 1 second end to end in this deployment.

Four screens, one investigation loop

● Dashboard

- Live composite anomaly signal
- Alert counts, severity mix
- Active defense rule count

● Attack Screen

- Launch DDoS / Scan / Botnet / MITM / HTTP Flood
- Pick detector + block strategy
- See group-level feature importance live

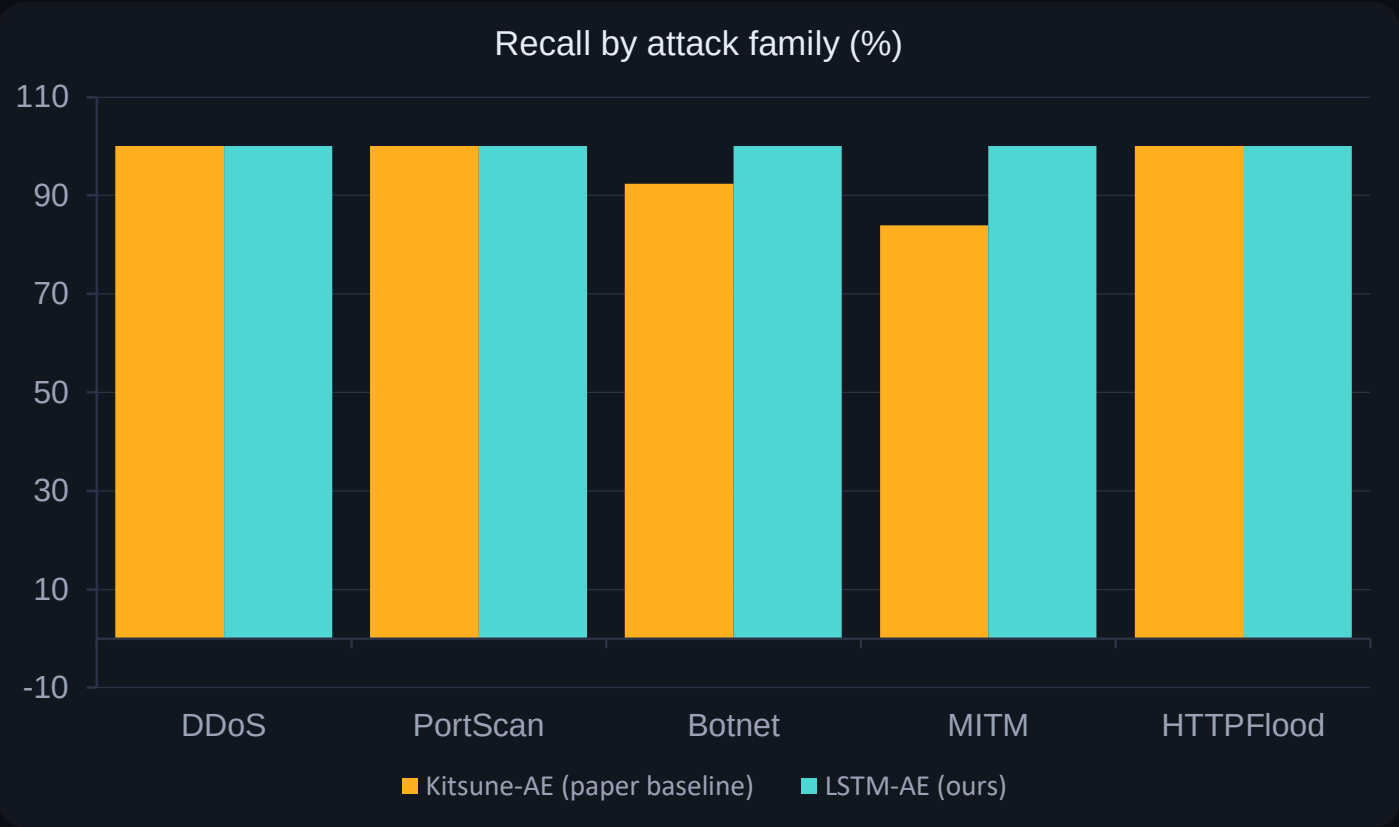
● Defence Screen

- All generated unified rules
- OpenFlow / iptables / Pfsense views
- Whitelist critical services (Sec 4.2)

● Model Lab

- Kitsune-AE vs LSTM-AE head-to-head
- Live fidelity (ADA-style) curve
- Per-attack recall comparison

LSTM-Autoencoder matches or beats the baseline on recall

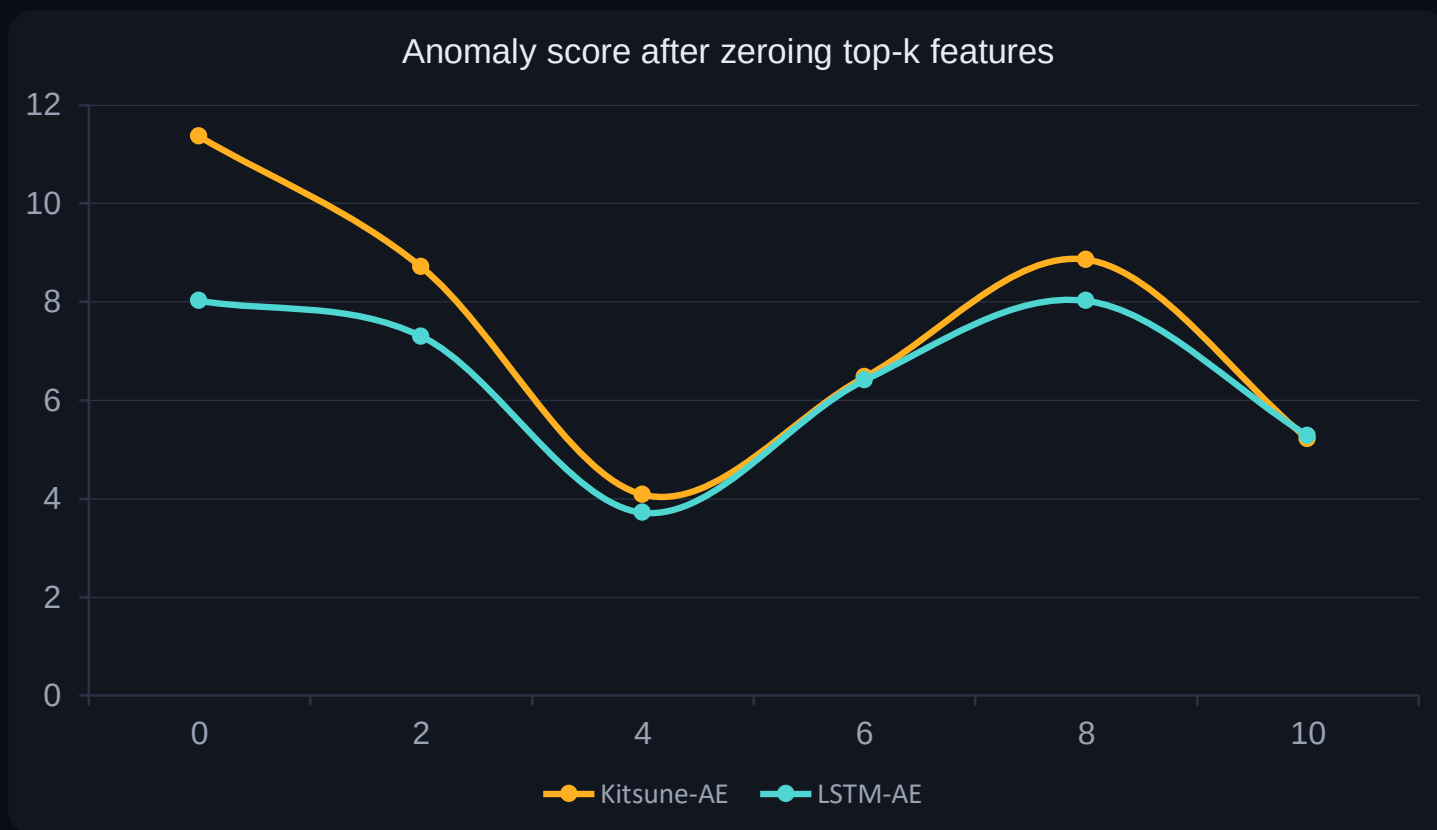


HEADLINE METRICS

Metric	Kitsune	LSTM-AE
Precision	100.0%	98.8%
Recall	95.3%	100.0%
F1	97.6%	99.4%
AUC	1.000	1.000

Measured live from the running app's /api/dashboard endpoints on synthetic, paper-inspired traffic.

Same fidelity test as the paper (Fig. 3), reproduced live



HOW TO READ THIS

We rank features by the XNIDS-style explainer, then zero the top k and re-score the same malicious sequence.

A steeper, lower curve means the explainer correctly identified the features the detector actually depends on.

Both curves fall sharply by k=4 — confirming the sparse-group-lasso explainer transfers cleanly across architectures.

What's real, what's simplified

● Real

- Full xNIDS-style explanation pipeline (history WRS + group-aware surrogate)
- From-scratch LSTM with manual BPTT, trained with real gradient descent
- End-to-end rule generation: scope → strategy → OpenFlow/iptables/Pfsense
- Working FastAPI + React app with persistence and live charts

● Simplified

- Synthetic, paper-inspired traffic generator (no internet access to CICIDS2017/NSL-KDD mirrors in this environment)
- ElasticNet surrogate instead of the paper's full sparse-group-lasso BCD solver (BCD version included for reference)
- Single-run fidelity test, not the paper's full multi-seed evaluation

Contributions

1

Member 1

Paper analysis & gap identification

- Read & summarized xNIDS (Sec 1-4, 6-7)
- Identified the history-modeling gap (Ch1)
- Wrote problem/solution/gap slides

2

Member 2

ML core: detectors & explainer

- Kitsune-style autoencoder baseline
- From-scratch LSTM-Autoencoder (BPTT)
- XNIDS-style explainer + rule generator

3

Member 3

Backend & data layer

- FastAPI routers, schemas, model store
- SQLAlchemy models (SQLite/MySQL)
- End-to-end API testing

4

Member 4

Frontend & documentation

- React console (Dashboard/Attack/Defence/Model Lab)
- SIGNAL design system
- README (HLD/LLD) & this presentation

Replace placeholder names with actual team members before submission.